

FinFlow

Expert-Learning Organizational Reasoning Engine

Project: INT-AI-01

Domain: AI / Knowledge Management / Engineering Productivity

A reasoning engine that answers organizational questions about a synthetic fintech organization by investigating its knowledge sources, drafts a first answer with a cited reasoning trace, learns from a human expert's ground-truth answer by distilling generalizable and leakage-free investigative patterns, and produces an improved second answer. The aim is to demonstrate learned *reasoning*, not memorized answers.

This document is self-contained: it covers the problem, the architecture, the design decisions and their rationale, the dataset, the learning and evaluation methodology, and an honest account of results. A reviewer should be able to assess the project from this document alone.

Contents

FinFlow

Expert-Learning Organizational Reasoning Engine

1. Overview and Problem

- Research status (stated honestly)

2. Requirements Coverage

3. Approach and Workflow

4. System Architecture

- Language model provider

- Retrieval

- Learning

- Evaluation

- Persistence

- Dashboards

5. Design Decisions and Rationale

- Generalizable patterns, not stored answers

- A deterministic leakage gate

- Held-out twin questions as the headline test

- BM25 retrieval rather than embeddings

- A pluggable LLM provider

- Hybrid evaluation with explicit gates

- A linear pipeline, no framework

- Synthetic dataset authored to exhibit the gap

- An honest results posture

6. Dataset Design

Cast and ownership

Timeline

Question families

Gold evidence sets

Worked example: P2 (duplicate-charge), end to end

7. Learning Mechanism

Leakage-free learning

Generalization, not memorization

8. Evaluation and Gates

9. Results

Verified (90 offline tests, zero API calls)

Directional (one live run, llama-3.1-8b-instant, family P2 to H2)

Unverified (not measured or did not complete)

Summary

10. Reproducibility and Setup

Local

Docker

What is and is not deterministic

11. Repository Structure

12. Limitations and Technical Debt

Limitations

Dataset and evaluation caveats

Runtime notes

Deliberate design choices (not debt)

13. Future Work

1. Overview and Problem

Organizations store critical knowledge across Slack conversations, issue trackers, wikis, and code repositories. Conventional AI assistants can retrieve facts, but they tend to miss the causal reasoning an experienced engineer applies when investigating an incident or a blocked milestone. Their answers therefore disagree with experts, and expertise stays trapped in individuals.

FinFlow addresses this with a human-in-the-loop learning loop. The system does not just retrieve harder; it learns reusable reasoning patterns from expert feedback and applies them to future questions. The headline test is generalization to a held-out twin question that the system never received feedback on, which separates a learned method from a memorized answer.

Research status (stated honestly)

The learning mechanisms are verified by an offline deterministic test suite. A directional validation on the `llama-3.1-8b-instant` model shows positive held-out transfer. The canonical `llama-3.3-70b-versatile` post-tuning evaluation has not completed because of Groq free-tier quota limits, so the central quantitative claim is not yet canonically validated. This document distinguishes verified, directional, and unverified results throughout.

2. Requirements Coverage

How the project addresses each part of the assignment.

Requirement	How it is met in FinFlow
Capture organizational questions	A Slack-style question feed of 9 questions across 6 categories, each tagged with a family, channel, and asker.
Investigate and draft an answer	The investigation agent retrieves evidence with BM25 and produces a V1 answer with a structured, cited reasoning trace and a confidence score.
Explainability	Every answer links each claim to specific evidence ids; retrieval snapshots (scores, matched terms, expansion flags) are persisted and inspectable.
Capture an expert (ground-truth) answer	Expert answers with root-cause rubrics and gold evidence sets are stored per question; the interactive dashboard accepts a typed expert answer live.
Gap analysis	A deterministic evidence diff (missed and extra evidence) plus an LLM-assisted reasoning and rubric gap analysis.
Learn from feedback	A learning-event generator distills generalizable retrieval signals and reasoning heuristics; a deterministic leakage gate guarantees no verbatim expert text is stored.
Produce a measurably improved answer	The agent generates a memory-augmented V2; V1 and V2 are compared on similarity, rubric coverage, evidence recall, and newly retrieved or cited evidence.
Prove reasoning, not memorization	Held-out twin questions (H1 to H3) test whether the learned method transfers to questions that never received feedback.
Dashboard for accuracy and learning trends	Two Streamlit dashboards: a read-only view over persisted runs and an interactive human-in-the-loop view.
Evaluation	A hybrid evaluation: versioned LLM judges plus deterministic metrics, with four explicit pass or fail gates.
Synthetic dataset	54 interlinked evidence items (Slack, tickets, wiki, commits) including 7 distractors, authored so answers require cross-source reasoning.
Reproducibility and packaging	90 offline deterministic tests, a pinned dependency set, and a single multi-mode Docker image.

3. Approach and Workflow

The pipeline is a plain linear sequence with no workflow or stage framework:

Question

- > Retrieval (BM25 + per-source balancing; learned expansion/routing in V2)
- > Investigation Agent (V1: answer + cited reasoning trace)
- > Human Answer (ground truth)
- > Gap Analysis (deterministic evidence diff + LLM reasoning/rubric gaps)
- > Learning Event Generator (generalizable patterns; leakage gate drops verbatim text)
- > Learning Memory (answer-free; relevance-filtered injection)
- > Investigation Agent (V2: memory-augmented; revises reasoning around new evidence)
- > Evaluation (LLM judges + deterministic metrics + four gates)

The loop in plain terms:

1. A question arrives from the organizational feed (a Slack-style channel).
 2. The investigation agent retrieves evidence and drafts a V1 answer with a cited reasoning trace and a confidence score.
 3. A human expert supplies the ground-truth answer.
 4. Gap analysis computes what V1 missed: missing evidence (deterministic) and missing reasoning or rubric points (LLM-assisted).
 5. The learning event generator distills generalizable patterns. A leakage gate guarantees no verbatim expert text is stored.
 6. Patterns are written to learning memory, then relevance-filtered when injected.
 7. The investigation agent produces V2 with the learned signals applied.
 8. Evaluation scores V1 against V2 using LLM judges and deterministic metrics, and computes four explicit gates.
-

4. System Architecture

The engine is organized into focused components, each with a single responsibility and a clear interface.

Language model provider

A pluggable `LLMProvider` interface. The implemented providers are `GroqProvider` (default model `llama-3.3-70b-versatile`) and `MockProvider` for offline tests. Only the LLM package imports a vendor SDK, which keeps the rest of the codebase provider-agnostic.

Retrieval

A `Retriever` interface with a deterministic BM25 implementation (no vector index). Retrieval applies per-source top-k balancing, a soft routing bias, and pluggable query expansion. Snapshots are persisted so retrieval is diagnosable after the fact.

For V2, an incident-only soft service-scope is applied: for production-incident questions, expansion-only evidence from a service other than V1's top result is down-ranked by a factor of 0.30 rather than dropped. Other question families are unaffected.

Learning

Distilled patterns are split into retrieval signals (expansion terms that widen retrieval) and reasoning heuristics (injected into the investigation prompt). A deterministic n-gram leakage gate drops any pattern field that shares a five-gram or longer with the expert answer. Relevance scoring gates which patterns transfer to a given question.

Evaluation

Versioned, fully logged LLM judges plus deterministic evidence overlap and improvement metrics. The active similarity judge awards graded partial credit and scores the share of key claims covered. A rubric-based root-cause judge scores each rubric element as hit, partial, or miss. The success signal blends rubric coverage and similarity rather than raw source retrieval.

Persistence

SQLite with nine tables: `questions`, `human_answers`, `answers`, `retrieval_snapshots`, `gap_analyses`, `learning_events`, `judge_results`, `evaluation_runs`, and `runs`. The read-only dashboard reads from this database and never writes to it.

Dashboards

A read-only dashboard presents persisted runs across the full pipeline. An interactive human-in-the-loop dashboard (default model `llama-3.1-8b-instant`) lets a user ask a question, run V1, type an expert answer, and generate V2, writing to the same SQLite database.

5. Design Decisions and Rationale

The decisions that shaped the system, with the reasoning and the tradeoff behind each.

Generalizable patterns, not stored answers

The system learns retrieval signals and reasoning heuristics, never the expert answer text. *Rationale:* the goal is transferable reasoning, so storing answers would be memorization and would not help on new questions. *Tradeoff:* learning is harder and less directly rewarded than caching answers, but it is the only honest way to demonstrate reasoning.

A deterministic leakage gate

A code-level n-gram check removes any learned field overlapping the expert answer, and the learning schema has no answer field at all. *Rationale:* the anti-memorization guarantee must be enforceable and testable, not a matter of prompt wording. *Tradeoff:* the gate is conservative and can drop borderline-useful phrasing, which is the safe direction.

Held-out twin questions as the headline test

Each primary question has a structurally similar twin that never receives feedback. *Rationale:* improvement on a trained question can be memorization; improvement on an unseen twin can only come from a transferred method. *Tradeoff:* it requires careful dataset authoring so that twins share method but not answers.

BM25 retrieval rather than embeddings

Retrieval uses lexical BM25 behind a `Retriever` interface. *Rationale:* it is fully deterministic, has no model dependency, and keeps the Docker image small and reproducible, which matters for an evaluation harness. *Tradeoff:* it misses pure-synonym matches, which is exactly the gap the learned expansion signals are designed to close, and the interface leaves a semantic retriever open for the future.

A pluggable LLM provider

A single interface with Groq and Mock implementations. *Rationale:* the engine should not be tied to one vendor, and tests must run offline and deterministically. *Tradeoff:* only one hosted provider is implemented, by design, to keep scope focused.

Hybrid evaluation with explicit gates

LLM judges are combined with deterministic metrics, and success is reduced to four pass or fail gates. *Rationale:* LLM judges alone are noisy and subjective, while deterministic metrics alone miss reasoning quality; gates make the success criteria explicit and auditable. *Tradeoff:* gate thresholds are heuristic rather than independently derived.

A linear pipeline, no framework

The orchestration is plain Python. *Rationale*: the flow is a fixed sequence, so a workflow framework would add dependencies and indirection without benefit, and a reviewer can read the whole flow in one file. *Tradeoff*: less dynamic branching, which this problem does not need.

Synthetic dataset authored to exhibit the gap

The corpus is constructed so each question has a piece of evidence reachable only through learned expansion. *Rationale*: the learning effect must be observable within a small, self-contained dataset. *Tradeoff*: the success story is partly a property of the authored data, which is stated plainly in the limitations.

An honest results posture

Results are labelled verified, directional, or unverified, and no success is declared on the central claim until a canonical run passes. *Rationale*: credibility depends on not overstating quota-limited evidence. *Tradeoff*: the headline claim remains formally open.

6. Dataset Design

The corpus is a synthetic fintech organization, deliberately interlinked so that answers require traversing more than one source. It contains 54 evidence items: 14 Slack threads, 14 tickets, 8 wiki pages, and 18 commits. Seven of those items are distractors that belong to no answer chain. There are 9 questions: 6 primary (P1 to P6) and 3 held-out twins (H1 to H3), plus 9 expert answers with root-cause rubrics and gold evidence sets.

Cast and ownership

Service	Team	Lead	Notes
Payment	Pay	Maya Chen	Charges, retries, settlement orchestration
Ledger	Books	Raj Patel	Records transactions; async settlement
Risk Engine	Sentinel	Sofia Alvarez	Inline fraud scoring; owned by Pay until Feb 2026
Notification	Comms	Tom Nguyen	Event-driven receipts and alerts

Timeline

January: async settlement ships (ADR-001 / commit a6). February: Risk Engine ownership moves Pay to Sentinel (OWN-100). April: Risk model v4 deploy (commit a13). May: the v2.5 settlement release slips two weeks (LED-412). June 3: duplicate-charge incident (commit a1 plus RISK-220 / commit a3). June 9: duplicate-notification incident (ADR-003 plus commit a4). June: Q3 planning, with fraud-scoring blocked on LED-430 and reconciliation blocked on an unassigned boundary (LED-450).

Question families

Question	Family	Held out	Theme
P1	release_delay	No	Why the May v2.5 settlement release slipped two weeks
P2	prod_incident	No	What caused the June 3 duplicate-charge incident
P3	milestone_blockage	No	What is blocking the Q3 fraud-scoring milestone
P4	service_ownership	No	Who owns the Risk Engine now and who owned it before
P5	design_decision	No	Why async settlement was chosen over synchronous
P6	service_ownership (contested)	No	Who owns the Payment-to-Ledger reconciliation boundary
H1	release_delay	Yes	Why the June notification-batching release is behind
H2	prod_incident	Yes	What caused the June 9 duplicate-notification incident
H3	milestone_blockage	Yes	What is blocking the Q3 settlement-reconciliation milestone

Each held-out twin shares the investigative method of its primary but differs in service, entities, and the nature of the blocker, so a memorized primary answer does not help.

Gold evidence sets

Evidence coverage is the size of the intersection of gold and cited evidence divided by the size of the gold set. The bold id is the one V1 typically misses, which drives the V1 to V2 recall gain.

Question	Gold evidence ids
P1	slack:rel-may-settlement, ticket:PAY-501, ticket:LED-412, commit:a9
P2	slack:inc-jun3-charge, ticket:PAY-530, ticket:RISK-220, commit:a3, commit:a1 , commit:a15
P3	wiki:q3-roadmap, ticket:RISK-240, ticket:LED-430 , slack:standup-risk
P4	wiki:ownership-matrix, ticket:OWN-100 , slack:ownership-risk-transfer
P5	wiki:adr-001-async-settlement, slack:design-async-settlement, wiki:adr-004-ledger-idempotency , commit:a6
P6	wiki:ownership-matrix, slack:recon-boundary , ticket:LED-450, commit:a14
H1	slack:rel-jun-notif-batch, ticket:NOT-300, ticket:NOT-320 , commit:a5
H2	slack:inc-jun9-notif, ticket:NOT-310, wiki:adr-003-notif-eventbus, commit:a4 , commit:a5
H3	wiki:q3-roadmap, ticket:LED-450, slack:recon-boundary , slack:standup-ledger, commit:a14

Worked example: P2 (duplicate-charge), end to end

Question: What caused the June 3 duplicate-charge incident? (family `prod_incident` , cross-source).

Root-cause rubric used by the judge: RC1, retry storm from Risk Engine timeouts (RISK-220 / a3). RC2, idempotency guard disabled (commit a1). RC3, mechanism: retries no longer deduplicated, so charges post twice. RC4, fix: revert the guard (PAY-540 / a15).

V1 (no memory): retrieval surfaces the symptom and the Risk timeouts and retries (RISK-220, a3), and may also surface the April model deploy (a13, a distractor). The actual root cause lives only in commit a1, whose message ("perf: disable idempotency check in the settlement hot path") has weak lexical overlap with "duplicate charge", so a1 ranks low and is missed. V1 concludes the proximate cause: Risk Engine timeouts triggered payment retries. This covers RC1 and partially RC3 but misses RC2 and RC4.

Gap analysis vs expert: missed evidence is commit a1 and commit a15; missed root cause is the disabled idempotency guard; the reasoning gap is that V1 stopped at the proximate cause and did not check recent Payment commits for a removed safeguard.

Learning event (sanitized, no verbatim answer): the heuristic "for incident root-cause analysis, check recent commits to the affected service for disabled or removed safeguards before blaming an upstream dependency"; a missed-evidence pattern with retrieval signals idempotency, guard, disable, revert; and routing from `prod_incident` to incident Slack, the RCA ticket, and recent commits.

V2 (memory-injected): routing boosts the commit source and query expansion adds idempotency, guard, and disable, so a1 and a15 are now retrieved. The heuristic in the reasoning prompt makes the model check recent commits. V2 names the retry storm, the disabled guard (commit a1), the deduplication mechanism, and the fix (PAY-540 / a15), covering RC1 to RC4 and citing the previously-missed commit a1.

This is a V1 to V2 improvement driven by a generalizable method, not a leaked answer.

7. Learning Mechanism

Leakage-free learning

The learning schema has no field for the expert answer. Before any pattern is stored, a deterministic gate removes any field that shares a five-gram or longer with the expert answer text. The result is that memory holds only generalized investigative hints and retrieval signals, never the expert's wording. This is asserted by the test suite and is the foundation of the anti-memorization claim.

Generalization, not memorization

The held-out twins are designed so that the primary answer is useless on the twin while the method transfers. For H2 (the twin of P2), the investigative move is identical: a visible platform cause (an at-least-once event bus) versus the real cause (a recent commit removed a safeguard). The heuristic "check recent commits for removed safeguards" and commit-log routing transfer exactly, even though the entities differ entirely (Notification, ADR-003, commit a4 instead of Payment, RISK-220, commit a1). Memorizing "disabled idempotency guard" would give the wrong specifics for H2; only the method helps.

8. Evaluation and Gates

Evaluation is hybrid: versioned LLM judges plus deterministic metrics. Per twin family it runs a training learning cycle (V1 to V2 on the train question), a held-out transfer test (baseline V1 versus post-learning V2 on the twin), and an ablation (V2 with learning stripped). It then computes four gates and persists an evaluation record.

Gate	What it checks	Threshold
Held-out generalization	V2 beats V1 on the held-out twin	> 0.10
Ablation attribution	Improvement is due to learning, not the V2 path alone	> 0.10
Same-question improvement	V2 beats V1 on the trained question	> 0.20
Leakage-free learning	No verbatim expert text stored	= 0.0

Deterministic metrics include evidence overlap and recall, the V1 to V2 deltas (newly retrieved and newly cited evidence), and evidence utilization (the share of newly retrieved evidence actually used in the final answer).

9. Results

Results are classified as verified (deterministic and reproducible), directional (a single non-canonical live run), or unverified (not yet measured).

Verified (90 offline tests, zero API calls)

- Leakage-free learning. The sanitization gate drops any pattern field sharing a five-gram with the expert answer; the learning schema has no answer field.
- Retrieval transfer. Learned generic signals and commit routing surface a previously missed commit on a held-out twin: H2 V1 excludes commit a4, H2 V2 includes it. The same holds for P2 commit a1.
- V2 consumes new evidence. Evidence utilization is greater than zero when V2 cites newly retrieved evidence.
- Relevance filtering. Service-specific patterns are rejected for an off-service twin while general patterns transfer, with diagnostics recorded.
- Gate and persistence plumbing. Four gates computed, evaluation record persisted, every judge call logged.
- Citation validation, confidence clamping, and deterministic retrieval and metrics.

Directional (one live run, llama-3.1-8b-instant, family P2 to H2)

Run after tuning, on a weaker non-canonical model, because the 70B quota was exhausted.

H2 metric	V1	V2	Ablation
Similarity	0.40	0.80	0.80
Rubric coverage	0.50	0.75	0.50
Evidence recall	0.20	0.40	0.60
Blended (similarity + rubric)	0.45	0.78	0.65
Evidence utilization	0.00	0.17	—

Held-out improvement on H2 is +0.325 blended (target above 0.10). The ablation delta is +0.125 (target above 0.10). Leakage passes (zero redactions of kept patterns, maximum overlap 0.00). Qualitatively, the newly-retrieved commit a4 was referenced in V2 reasoning, and the V2 answer correctly named a recent consumer refactor that dropped the deduplication step, which V1 had not. The transferred heuristic was generic.

Unverified (not measured or did not complete)

The canonical central claim on the 70B model across all held-out families was not measured. The full evaluation on `llama-3.3-70b-versatile` could not complete because the Groq free-tier daily token cap was exhausted.

The only completed canonical run was before tuning and with the more lenient similarity judge. It failed three of the four gates:

Gate	Value	Threshold	Result
Held-out generalization	0.021	0.10	FAIL
Ablation attribution	0.013	0.10	FAIL
Same-question improvement	-0.151	0.20	FAIL
Leakage-free learning	0.0	0.0	PASS

The root cause of that failure was that retrieval improved but reasoning did not follow, and the lenient judge left no headroom for V1. The tuning work targeted exactly these issues, and the directional 8B run shows the fix works on 8B, but this has not been reconfirmed on 70B. The pre-tuning failure has therefore not been superseded by a passing canonical run.

Summary

The mechanisms are verified. The quantitative central claim is directionally supported on 8B for one family and unverified on the canonical 70B model and the full held-out set. No success on the central claim should be declared until a canonical 70B run passes.

10. Reproducibility and Setup

Local

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # set GROQ_API_KEY (not needed for tests)
PYTHONPATH=. pytest          # 90 offline tests
PYTHONPATH=. python scripts/seed_db.py
PYTHONPATH=. python scripts/run_pipeline.py --question P2
PYTHONPATH=. streamlit run dashboard/app.py          # read-only dashboard
PYTHONPATH=. streamlit run dashboard/live_app.py     # interactive dashboard
```

Requires Python 3.11 or later. Verified dependency set: groq 1.4.0, rank-bm25 0.2.2, streamlit 1.37.1, pandas 2.2.2, pydantic 2.8.2.

Docker

One image, four modes, no secrets baked in. Secrets are injected only through the environment or a `.env` file, which is excluded from the image.

```
cp .env.example .env          # set GROQ_API_KEY
docker compose build
docker compose --profile tests run --rm tests          # 90 tests, offline
docker compose up app          # dashboard at http://localhost:8501
docker compose --profile eval run --rm eval           # evaluation (needs key and quota)
```

Key environment variables: `FINFLOW_PROVIDER` (groq or mock, default groq), `GROQ_API_KEY`, `FINFLOW_MODEL` (default llama-3.3-70b-versatile), `FINFLOW_JUDGE_MODEL`, `FINFLOW_DB`, `FINFLOW_RETRIEVAL_K` (default 4), and `PORT` (default 8501).

What is and is not deterministic

The full test suite is deterministic: it uses mock and scripted providers with no network, clock, or randomness. BM25 retrieval is deterministic given the corpus, with stable tie-breaks by source id. All deterministic metrics and the leakage gate are reproducible.

Live LLM outputs are not byte-reproducible; investigation, gap, learning, and judge calls vary run to run even at temperature zero because of provider-side nondeterminism. Prompt and judge versions are recorded on every artifact so runs remain interpretable.

11. Repository Structure

```
finflow-reasoning-engine/
  requirements.txt  pyproject.toml  .env.example  .gitignore
  FinFlow_Documentation.pdf      this document
  finflow/              42 Python files
    config.py  orchestrator.py
    models/    answer, evaluation, evidence, gap, human_answer,
               learning, question, reasoning, retrieval
    llm/       base, groq_provider, mock_provider, factory
    retrieval/ retriever, bm25_retriever, expansion, knowledge_store,
               loaders, snapshot_store
    agents/    investigation_agent, gap_analysis_agent,
               learning_event_generator, json_parsing
    memory/    learning_memory
    evaluation/ judge, evaluator, evidence_overlap, improvement, leakage
    persistence/ repository, sqlite_repo
    prompts/   investigation_v1/v2, gap_v1, learning_extract_v1/v2,
               judge_similarity_v1/v2/v3, judge_root_cause_v1

  data/
    feed.json          9 questions
    human_answers.json 9 ground-truth answers + rubrics + gold evidence
    sources/           slack (14), tickets (14), wiki (8), commits (18)
  scripts/  run_pipeline.py  run_evaluation.py  seed_db.py
  dashboard/ data.py  app.py  live_app.py  ui.py
  tests/    10 files, 90 tests
```

Category	Count
Python source files	42
Prompt templates	9
Scripts	3
Dashboard modules	5
Test files	10
Tests	90 (all passing, offline)
Evidence items	54
Questions	9 (6 primary, 3 held-out)
SQLite tables	9

12. Limitations and Technical Debt

Limitations

- The central research claim is not canonically validated. The 70B evaluation is quota blocked, so all live numbers are 8B, single-sample, and from one human-proxy rater across three twin families. They are directional, not statistically robust.
- The incident-only service-scope filter is marginal at a factor of 0.30. The family gate, not the down-rank strength, prevents the earlier P4 regression, so the penalty can be strengthened later without P4 risk.
- Base-matched cross-service contamination is unaddressed. The filter only down-ranks expansion-only items; off-service evidence matching the question's own terms still surfaces.
- The similarity judge is mildly over-generous on partially-wrong answers, a large improvement over the previous version but still tunable.
- Out-of-distribution questions can confabulate; there is no abstention path, and the tokenizer treats related word forms as distinct.
- The mock provider is a deterministic test harness, not a functional offline demo.

Dataset and evaluation caveats

- The corpus is authored to exhibit the learning gap. Two items were reworded so the missed evidence stays outside the question's lexical reach: ticket PAY-540 (so the mechanism lives only in commits a1 and a15) and commit a4 (so the H2 query cannot retrieve it in V1). This is legitimate synthetic design but means the success story is partly a property of the authored data.
- LLM-as-judge similarity is subjective and noisy, though logged and versioned. Root-cause scoring uses a fixed rubric.
- Evidence utilization counts cross-domain noise in its denominator, so it understates true utilization.
- Gate thresholds and the relevance threshold are heuristic, chosen as reasonable targets rather than independently derived. They were not tuned to manufacture success.
- The sample is three twin families, so aggregate gate values are means over a tiny sample.
- The ablation path is close to the V1 baseline, so the ablation and generalization gates are correlated rather than fully independent.

Runtime notes

- Scripts require `PYTHONPATH=.` ; Docker normalizes this.
- The Groq free tier exhausts its daily token budget quickly, which blocks the canonical evaluation. Directional validation used the separate per-model quota of the 8B model.
- There is no retry or backoff on Groq calls, so a transient rate-limit error aborts a run.

Deliberate design choices (not debt)

- A single hosted provider (Groq) plus Mock, by design.

- BM25 over embeddings, for determinism and Docker simplicity, with the `Retriever` interface left open for a semantic swap-in.
-

13. Future Work

- Run the canonical 70B post-tuning evaluation once quota is available.
- Expand the number of families and categories for statistical robustness.
- Add retry and backoff for transient API errors.
- Add an optional semantic retriever behind the existing `Retriever` interface.
- Sharpen retrieval-signal specificity to raise the evidence-utilization signal-to-noise ratio.